A function that's placed `around` another executes code both before and after it:

```
withOpenDb = around (dbActivity) ->
  openDbConnection()
  dbActivity()
  closeDbConnection()
```

Now you can use `withOpenDb` everywhere:

```
getUserData = withOpenDb (users) ->
  users.get 'user123'
```

`Around` doesn't just get invoked and discarded: it can control whether or not the function it wraps is invoked *at all*. If `openDbConnection` evaluates to `false` when the database fails to open, then the database activity can be skipped:

```
withOpenDb = around (dbActivity) ->
  if openDbConnection()
    dbActivity()
    closeDbConnection()
```

That makes `around` quite powerful *and dangerous*. It's possible to implement combinators that have even more power (and that are even more dangerous), such as modifying the arguments to the wrapped function and controlling the return value of the created function.

Functions are often used as methods in CoffeeScript, so generic functions that create other functions need to work correctly when those functions are methods. What happens when these combinators are used with methods?

6.4.4. Working with objects

The tricky thing about method objects is referencing `this` (a.k.a. @). Suppose you have a program that controls the movement of a toy robot. The program must start the toy robot engine before making it move. The program also needs to stop the engine once it is done moving the toy robot. This is essentially the same problem as the previous database example,